# Reverse入门指北 2026

2026 @nkanf @fanchai

## 逆向工程是什么？

其实你可能一直都在跟逆向工程打交道。不论是你在某些论坛下载的“破解版”软件，在风灵月影宗得到的无敌修改器（或者是游戏里突然飞天遁地的大哥），还是你在 MC，缺氧，星露谷，黑神话，杀戮尖塔2……等等几乎一切游戏里玩到的各种模组，一些软件的“增强包”，“扩展组件”……等等等等这些去拆解和改变软件原有逻辑的东西，都离不开逆向工程！

简单来说，逆向工程就是用各种手段拆解一个程序，搞懂它的逻辑，然后在这个程序里面找到自己想要的东西，或者进一步按照自己的想法去改造这个程序。

在 CTF 的背景下，逆向工程方向的题目都是在程序里面埋藏一个 flag，你需要想尽一切办法拆解这个程序以弄懂它的逻辑，最后解出这个 flag！

## 逆向工程可以做什么？

从上面列出的一些例子也可以看出来，逆向工程或许是 CTF 里面最具有实用价值的一个方向。

一旦你学会了逆向工程：

- 下载到的软件遇到各种限制，只能干着急？不！你还可以先对它 ~~学习研究一~~ 番～喜欢用的工具没有自己想要的功能，只能忍痛换一个？不！你还可以自己去加～
- 不知道别人发给自己的软件安不安全，只能相信他？不！你可以自己去拆解～

想在游戏里玩点别的花样，但是找不到模组，只能作罢？不！你可以自己做一个～可以说，有了逆向工程，程序对你来说就再也不是一个黑盒了，你可以随心所欲地对它做几乎任何事情！

## 我可以怎么学习逆向工程呢？

~~直接去做签到题，干就完子~~

工欲善其事，必先利其器！一个好用的工具对逆向分析至关重要（你也不想直接从机器代码开始分析吧）。

很多程序是直接以二进制的形式存在计算机里的，形如 10110100……，这就是机器代码

### 我们第一个介绍的是：反编译工具

反编译工具可以帮助我们把程序从难懂的机器代码提升到更容易读的层次。

对于不同语言编写的程序，我们也会使用不同的反编译工具，因为他们的编译流程不一样，产物的类型也不一样。

编译是指把一个程序从源代码变成可执行产物的过程

关于编译，这里我们再引入一个小小的概念，JIT（Just In Time）编译和 AOT（Ahead Of Time）编译。

- 在 JIT 编译里面，产物是由中间代码组成的，在运行的时候，运行环境会一边将中间代码编译为机器代码，一边去执行。
    - 中间代码是介于源代码和机器代码之间的一种表示。比源代码难懂，但仍然具有一定的可读性。

在 AOT 编译里面，产物就直接由编译后的机器代码组成。但是像一些脚本语言，比如 Python、JavaScript，它们通常不会被编译成二进制产物，而是直接被解释执行。

- 解释是指由一个被称为解释器的特殊程序，将源代码（或中间代码）逐行（或逐句）翻译成机器码并立即执行的过程。

不要被上面这么多概念吓到！其实提到这么多概念只是为了说明：我们对于不同语言编写的程序，进行逆向工程的策略也不一样。下面再简单介绍几个常见语言的情况，给它们归个类：

- C 或 C++ 编写的程序，会被编译成机器代码。所以进行逆向需要从机器代码的层面开始。
- Python 编写的程序分为多种情况。如果是被解释执行的，那么我们直接可以看到它的源代码，或者用一些方法解包之后看到它的源代码。但是如果是被编译的。那也需要从机器代码的层面开始逆向。
- Java 编写的程序也分为多种情况，但是我们一般遇到的都是 JIT 编译的 Java 程序。这个情况下，它是以字节码的形式存在，并且通过 JVM 运行环境来执行。对它进行逆向，只需要从字节码的层面开始。
- C# 编写的程序同样分为两种情况。大多数的 C# 程序是以 JIT 编译的。这个情况下，它是以 MSIL 的形式存在，通过 CLR 运行环境来执行。对它进行逆向。只需要从 MSIL 的层面开始。
    - 关于字节码和 MSIL，这里我们只需要知道它们都属于中间代码即可，感兴趣的同学可以自己去查询相关介绍。
        - 字节码 (Bytecode) 是一种介于源代码和机器码之间的中间代码，通常由 Java 等语言编译生成并交由虚拟机（如 JVM）来运行。参考链接：https://en.wikipedia.org/wiki/Bytecode
        - MSIL (Microsoft Intermediate Language) 是一种微软 .NET 平台专属的中间代码，通常由 C# 等语言编译生成并交由 CLR 运行环境来执行。参考链接：

知道了程序存在的形式之后，我们就可以选用对应的反编译工具进行反编译。

但是话又说回来，在电脑上看见一个程序 `.exe`，我怎么知道它是用什么语言编写的呢？其实可以使用专门的工具来进行检测。这里我们常用的有 exeinfope 和 DIE 等等。将程序拖进这些检测工具里面，你就能看到它是用什么语言编写的。

- Exeinfo PE 是一款专门用于检测程序编写语言、编译方式及是否加壳的识别工具。参考链接：https://github.com/ExeinfoASL/ASL

DIE (Detect It Easy) 是一款跨平台的综合文件检测工具，能够快速识别可执行文件的类型、编译器以及加壳状态。参考链接：https://github.com/horsicq/Detect-It-Easy

这样，我们直接对号入座即可。如果是以机器代码形式存在的程序，那么我们有通用的反编译工具，比如著名的 IDA 或者 Ghidra。

- IDA 是一款通用的反汇编与反编译工具，主要用于将编译后的机器代码还原成汇编代码和可读性更高的伪代码。参考链接：https://hex-rays.com/
- Ghidra 是一款免费开源的通用逆向工程框架，同样具备将机器代码反编译并提升为伪代码的功能。参考链接：https://ghidra-sre.org/

如果是以中间代码形式存在的程序，那么对于不同的中间代码，我们有不同的反编译工具。例如，对 Java 程序来说，我们可以用 JADX 进行反编译。而对于 C# 程序来说，我们可以用 dnSpy 进行反编译。

JADX 是一款专门针对 Java 和 Android 程序的反编译工具，能够将字节码还原成直观的 Java 源代码。参考链接：https://github.com/skylot/jadx

- dnSpy 是一款针对 C# 和 .NET 程序的专属工具，不仅能将 MSIL 中间代码反编译为 C# 源码，还内置了动态调试功能。参考链接：https://github.com/dnSpyEx/dnSpy

反编译工具的最主要作用就是代码提升，它可以将晦涩难懂的机器代码或中间代码，翻译并还原成人类更容易理解的伪代码（通常是 C/C++ 源代码的风格）。

下面用一张图来更方便理解：

未知目标程序 .exe / 二进制
文件
查壳与语言检测
工具: exeinfope, DIE
程序的存在形式?
中间代码 / JIT编译
中间代码
机器代码 / AOT编译解释执行
介于源码与机器码之间
机器代码 Java 程序 C# 程序
源代码 / 脚本
10110100... 形式: 字节码形式: MSIL
代表: Python, JavaScript
代表: C/C++, 编译的Python 环境: JVM 环境: CLR
通用反编译工具针对性反编译工具针对性反编译工具
解包工具或直接阅读
IDA, Ghidra JADX dnSpy
代码提升
还原为人类易读的伪代码
C/C++ 风格

但是有的时候，面对复杂的代码库，直接阅读反编译的结果可能会让人摸不着头脑。这个时候，我们可以换一个方向，动态地去追踪代码的执行过程。

我们第二个介绍的是：动态调试工具首先上面提到的很多反编译器本身也同时带有动态调试功能，可以直接使用这些工具自带的动态调试。

不过也有一些专用的动态调试工具，比如 Windows 上的 x64dbg，Linux 上的 GDB，可以面向移动端的 Frida 等等。

- x64dbg 是一款用于 Windows 平台的动态调试工具，可以通过控制程序运行来观测底层机器指令的执行状态。参考链接：https://x64dbg.com/
- GDB 是一款主要在 Linux 系统下使用的命令行动态调试工具，用于在程序运行时追踪和分析内部逻辑。参考链接：https://www.gnu.org/software/gdb/

Frida 是一款跨平台的动态插桩工具，允许通过注入脚本的方式直接在程序运行时拦截和观测内部函数的状态。参考链接：https://frida.re/

动态调试的技巧有很多，这里就不详细介绍了。不过如果说静态分析是在空间的维度上拆解程序的方方面面，那么动态分析就是在时间的维度上观测程序某个地方的前世今生。举一个例子，假如你正在分析一个 VIP 检测函数，但是这些函数错综复杂，不容易看出来它们是谁调用的，最终又会执行到哪里。这个时候就可以用动态调试的方法，把程序运行起来，直接去观测这些信息。

## 我们第三个介绍的是：其他各种辅助工具

在有了静态反编译和动态调试两大法宝之后，理论上我们就可以分析任何程序了。但是实际上为了提高效率，我们还会搭配各种很多辅助工具，它们可能是一些脚本、插件、工具箱等等，通过对一些手动的流程进行自动化、复用经验等的方式，来帮助我们更便捷地进行逆向工程！比如前面有提到的查壳/检测工具（exeinfope，DIE等）。

## 正确看待并使用工具

介绍了这么多工具之后，我们还有一个非常重要的问题，那就是：如何正确地使用这些工具？

在 ~~（黑客）网~~ 络安全的世界里，一直存在着一群被叫做「脚本小子」的人。他们的特征就是极度依赖工具，对于任何问题的解决方法都只有“拖进工具看看”，而一旦工具出了问题、流程跑不通，就会束手无策。

我们当然不能变成这样的人，也就是说，我们需要「掌握」工具，而不是「依赖」工具。这说起来可能有点抽象，但是举几个例子就能明白：

- 在使用「反编译一键识别密码算法」工具的时候，我们还需要在发现它有识别盲区之时，有自己分析的能力，或者更进一步的，还能够想办法去改进识别算法，改进这个工具，覆盖更多的范围…
- 在使用「一键过反调试插件」工具的时候，我们还需要在它不能过的时候，有自己手动分析的能力，然后进一步地去改进工具…

也就是说，对于工具，我们一方面要致力于「知其然，更知其所以然」，另一方面还要致力于「从“使用者”进化为“创造者”」，先有利器，后练内功，这样才能不断进步。

# AI Agent，逆向工程的强大新工具……

在纷乱的互联网上，大家可能已经被 AI 搅乱了心神，或者听说过各种危言耸听的话语。但是请记住人的主体性

## 对于这次学习的目的——使用 AI：

当前AI技术不断升级，使用AI辅助学习CTF可以让你事半功倍。很多逆向工程问题甚至可以直接丢给AI，然后等着得到结果。事实上，对于moectf2026中的大多数逆向工程挑战确实如此！直接使用AI解题虽然能拿到分数，但是我们必须强调，在这场新生赛中，这并不是值得鼓励的行为！

在新生赛中，能学到东西是第一，而拿分是第二。我们出题的目的是渐进式的引导大家学习这个方向，而不是为了挑战大家。

我们希望你在这场比赛中学习知识，享受解决CTF挑战的快乐，并逐步构建一套行之有效的 CTF 解题“方法论”，由此来达到真正意义上的“入门”CTF。更具体点讲，我们鼓励你使用AI做这些辅助：

- 询问AI关于工具下载和使用的问题
- 询问AI关于编程语言语法的问题
- 遇到报错信息时，及时求助AI
- 遇到看不懂的概念问题时，及时求助AI
    - 在求助 AI 的时候，我们不推荐直接让 AI 给出答案我们推荐使用苏格拉底提问法来更好的完成学习的过程
    - 你可以请求 AI 使用苏格拉底提问法来教你东西，例如“请使用苏格拉底提问法教我汇编指令。”
    - 我们很清楚现在 AI 的能力，它可以把这些题目全部一把梭，这是预期内的事情。所以，如果你想学到东西的话，请不要这么做。

## 而在这之后呢——驾驭 AI：

我们迎来了一个被 AI 工具席卷的时代。在不断迭代的 Agent 面前，很多逆向工程问题都可以简单丢给它然后等着得到结果。

但是这并不意味着我们不用学习逆向工程了，正如前文所述，AI Agent 也还是一个更加强大的工具。它依旧有盲区，有很多可以改进的地方。而如果要改进它，则更需要我们拥有丰富的逆向工程经验、知识与技术，例如：

- 给现有的逆向工具设计更好用的 MCP 接口，或者制作更 AI 友好的逆向工具，比如进行 CLI 化，脚本化…
- 把各种各样的经验沉淀成 Skills，或者对 Skills 进行优化迭代，想办法让它效果更好… 结合前沿的 Loop 工程、Harness 工程等，为逆向工程这个场景设计出更好的 Agent 方案…
- 所以希望大家能够在有 AI 辅助的时代，能够以更高的效率学习逆向工程，而非成为 AI 时代的「脚本小子」。

# 现在就开始吧！

逆向工程入门指北2026.exe

# 附录

这里分享一些可能有用的资源

## AI Agent 类

- Codex / Claude Code
- Awesome-RE-MCP
- reverse-skill

## 搜索类

- 各种 AI
- Google
- Bing

## 论坛类

- 吾爱破解论坛
- 看雪论坛

## 资源类

- retoolkit （逆向工具大杂烩）
- 爱盘